

Guía del Backend Java

Comprendiendo conceptos clave con ejemplos de la vida real

Propósito

Esta guía tiene como objetivo aclarar conceptos clave del desarrollo backend en Java usando analogías cotidianas, ejemplos sencillos (como un restaurante), y una guía clara de cuándo usar cada herramienta, patrón o tecnología del stack.

Capítulo 1: Servicios REST y HTTP

Explicación técnica

REST (Representational State Transfer) es un estilo arquitectónico para diseñar servicios web. Utiliza el protocolo HTTP para exponer recursos (datos) mediante operaciones estándar como GET, POST, PUT, DELETE.

Analogía: La carta del restaurante

Imagina que estás en un restaurante:

- El menú es la documentación de tu API (Swagger).
- El mesero es el endpoint que recibe tu solicitud (ej. GET /menu).
- La orden es tu petición HTTP.
- El cocinero es la lógica de negocio que procesa tu pedido.
- El plato servido es la respuesta del backend.

¿Cuándo sí usar REST?

- Cuando necesitas exponer servicios a un frontend (React, Angular).
- Para integrarte con otros sistemas.
- Si tu aplicación usa microservicios que se comunican entre sí.

¿Cuándo no usar REST?

- Para lógica interna entre clases o módulos (usa servicios Java puros).
- Si tu aplicación es 100% batch y no recibe peticiones HTTP.

Errores comunes

- No manejar errores (no enviar HTTP 400, 404 o 500 cuando corresponde).
- No validar las entradas.
- Usar verbos incorrectos (ej. usar GET para crear recursos).

Capítulo 2: Spring Boot

Explicación técnica

Spring Boot es un framework que simplifica la creación de aplicaciones Java al integrar y configurar automáticamente muchas dependencias comunes.

Analogía: Cocina moderna con recetas listas

Spring Boot es como una cocina moderna equipada con hornos inteligentes, recetas automáticas y sensores que hacen tu trabajo más fácil. No necesitas configurar todo desde cero: Spring ya lo hizo por ti.

¿Cuándo sí usar Spring Boot?

- Cuando construyes APIs REST en Java.
- Si quieres rapidez en el desarrollo y enfoque en la lógica de negocio.
- Si estás trabajando con microservicios.

¿Cuándo no usarlo?

- Si estás construyendo una librería o herramienta de bajo nivel sin necesidad de un servidor embebido.
- En apps muy pequeñas donde una dependencia tan robusta es innecesaria.

Errores comunes

- No entender qué está haciendo Spring por ti (inversión de control, autoconfiguración).
 - Sobrecargar el proyecto con muchas dependencias.
 - No separar responsabilidades en los controladores.
-

Capítulo 3: Pruebas Unitarias (JUnit, Mockito, JaCoCo)

Explicación técnica

Las pruebas unitarias validan que las unidades más pequeñas del código (métodos o clases) funcionen correctamente de forma aislada.

Analogía: Probar el plato antes de servirlo

Un buen chef siempre prueba el plato antes de llevarlo al cliente. Las pruebas unitarias son ese momento donde verificas que el sabor (lógica) está bien antes de exponerlo al mundo.

¿Cuándo sí escribir pruebas unitarias?

- Para lógica de negocio (cálculos, validaciones, reglas).
- Antes de refactorizar código existente.
- Cuando usas TDD (Test Driven Development).

¿Cuándo no es prioridad?

- En configuraciones triviales sin lógica propia.
- En código de prueba o exploratorio que será descartado.

Errores comunes

- No probar los casos límite o errores.
 - Hacer pruebas frágiles dependientes de datos externos.
 - Escribir pruebas sin assertions claras.
-

Capítulo 4: Arquitectura Hexagonal y DDD

Explicación técnica

La arquitectura hexagonal (o Ports and Adapters) separa la lógica de negocio de la infraestructura. DDD (Domain Driven Design) ayuda a modelar el software con base en el lenguaje del negocio.

Analogía: Cocina con roles separados

En un restaurante bien organizado:

- Los chefs solo cocinan (dominio).
- Los meseros no cocinan, solo reciben pedidos (adaptadores).
- Los puertos son las ventanas por donde entra y sale la comida.

¿Cuándo sí usar esta arquitectura?

- En proyectos medianos o grandes que crecerán con el tiempo.
- Cuando necesitas claridad, escalabilidad y mantenibilidad.

¿Cuándo no usarla?

- En MVPs rápidos o proyectos experimentales.
- Si el equipo no está familiarizado y genera más complejidad.

Errores comunes

- Poner lógica de negocio en los controladores.
 - No respetar la separación entre capas.
 - Usar "adaptadores" que se conectan directo al dominio sin interfaz.
-

Capítulo 5: CI/CD (Integración y Despliegue Continuo)

Explicación técnica

CI (Continuous Integration) automatiza la validación del código nuevo. CD (Continuous Delivery/Deployment) automatiza el paso del código probado hasta producción.

Analogía: El robot que cocina y sirve sin que tú estés

CI/CD es como tener un robot que al detectar una nueva receta, prueba los ingredientes, cocina, empaqueta el pedido y lo entrega al cliente, todo sin intervención humana.

¿Cuándo sí usar CI/CD?

- En cualquier proyecto activo con despliegues frecuentes.
- Cuando hay varios desarrolladores colaborando.
- Si se quiere reducir errores humanos y acelerar la entrega.

¿Cuándo no usarlo?

- En prototipos sin despliegue real.
- En pruebas personales sin impacto en producción.

Errores comunes

- No validar el código antes del build.
 - No separar ambientes (dev, test, prod).
 - No monitorear los pipelines.
-

Capítulo 6: Despliegue en la Nube (AWS)

Explicación técnica

AWS ofrece servicios como EC2, S3, ECS y Lambda para alojar aplicaciones en la nube, con escalabilidad, monitoreo y alta disponibilidad.

Analogía: Alquilar una cocina profesional

Desplegar en la nube es como alquilar una cocina equipada para eventos grandes. No la construyes tú, solo la usas y pagas por el tiempo y uso que le des.

¿Cuándo sí usar AWS?

- Cuando tu aplicación debe estar en línea para usuarios reales.
- Si necesitas escalar rápidamente según la demanda.
- Para pruebas integradas con servicios reales.

¿Cuándo no usarla?

- En entornos de desarrollo local.
- Si aún no tienes funcionalidades completas.

Errores comunes

- No usar variables de entorno para claves.
 - Dejar servicios corriendo sin control de costos.
 - No monitorear logs o métricas.
-

Capítulo 7: Principios SOLID

Explicación técnica

Los principios SOLID son cinco reglas que ayudan a escribir código más mantenible, escalable y fácil de entender:

- **S**: Single Responsibility Principle
- **O**: Open/Closed Principle
- **L**: Liskov Substitution Principle
- **I**: Interface Segregation Principle
- **D**: Dependency Inversion Principle

Analogía: Cocina donde cada quien tiene una sola tarea clara

Un chef que sólo cocina postres no debería también lavar platos. Cada rol tiene una única responsabilidad clara, y el restaurante mejora en eficiencia.

¿Cuándo sí aplicar SOLID?

- Siempre que escribas clases, servicios o componentes.
- En sistemas colaborativos o de largo plazo.

¿Cuándo no obsesionarte?

- En prototipos rápidos donde prima la velocidad.
- Si causa sobrearquitectura innecesaria en tareas simples.

Errores comunes

- Tener clases que hacen de todo (violación del SRP).
 - Interfaces con métodos que no se usan (violación ISP).
 - No usar inyección de dependencias (violación DIP).
-

Capítulo 8: Documentación de APIs con Swagger

Explicación técnica

Swagger es una herramienta para documentar APIs RESTful de forma visual, interactiva y estandarizada.

Analogía: Menú ilustrado con fotos y descripciones

Swagger es como un menú donde puedes ver fotos del plato, ingredientes, tiempos de espera y opciones adicionales. Facilita que el cliente (frontend o integraciones) sepa exactamente qué pedir.

¿Cuándo sí usar Swagger?

- Siempre que crees un API REST.
- Si tu API será consumida por otros equipos.
- Para facilitar pruebas manuales e integraciones.

¿Cuándo no es prioritario?

- En APIs internas desechables o pruebas rápidas.
- En funciones privadas sin exposición externa.

Errores comunes

- No documentar errores posibles (401, 403, 500).
 - No versionar la API.
 - Usar nombres técnicos confusos o genéricos.
-

Capítulo 9: DevOps como Cultura

Explicación técnica

DevOps no es una herramienta, sino una **cultura de colaboración** entre desarrollo (Dev) y operaciones (Ops) que promueve la entrega continua, automatización, monitoreo y mejora constante del software.

Analogía: Restaurante con cocina y meseros en sincronía

Un restaurante exitoso no solo tiene buenos cocineros (Dev), sino que también los meseros (Ops) entienden el menú, ayudan a entregar los platos a tiempo, resuelven problemas y colaboran con la cocina. Si no se comunican, hay retrasos, confusión y errores en los pedidos.

¿Cuándo sí adoptar DevOps?

- En proyectos que crecen rápido y necesitan despliegues frecuentes.
- Cuando hay necesidad de escalar sin perder calidad.
- En equipos que desean mejorar velocidad, estabilidad y trazabilidad.

¿Cuándo no obsesionarse?

- En equipos de 1 sola persona o sin infraestructura compleja.
- Si no hay necesidad de múltiples entornos ni usuarios externos.

Errores comunes

- Pensar que DevOps es solo instalar Jenkins.
 - No compartir el conocimiento entre devs y ops.
 - Ignorar la observabilidad (logs, métricas, alertas).
-

Capítulo 10: La historia de ARKA – Un backend bien cocinado

Contexto

ARKA es una plataforma de abastecimiento y gestión de pedidos. El equipo que la construye tiene una misión clara: entregar una API robusta, escalable, segura y lista para producción. Esta es la historia de cómo aplicaron todos los principios de esta Biblia.

Paso 1: Modelar la cocina (Arquitectura Hexagonal + DDD)

Lo primero fue organizar la cocina. Separaron claramente:

- El dominio (la lógica de negocio): manejar pedidos, inventario, pagos.
- Los puertos (interfaces) que definían qué debía hacerse.
- Los adaptadores: uno REST para recibir pedidos, otro para guardar en base de datos, y uno más para integrarse con el sistema de pagos.

Así, los chefs (dominio) no hablaron directo con los clientes (REST) ni con el banco (infraestructura). Todo estaba encapsulado.

Paso 2: Servir con REST + Spring Boot

Crearon endpoints REST con Spring Boot, usando `@RestController`, validaciones, DTOs y control de errores estandarizados (`@ControllerAdvice`). El cliente pedía `/api/pedidos` y el sistema respondía limpio, como un buen mesero.

Paso 3: Probar los platos (JUnit, Mockito, JaCoCo)

Cada nueva funcionalidad iba acompañada de pruebas:

- JUnit para la lógica de negocio.
- Mockito para simular el acceso a datos.
- JaCoCo para ver la cobertura de código.

Solo cuando todo estaba verde, se pasaba al siguiente paso. Nada salía sin probarse antes.

Paso 4: Automatizar la cocina (CI/CD)

Usaron GitHub Actions. Cada vez que alguien hacía un cambio:

- Se corrían las pruebas.
- Se construía el artefacto.
- Se desplegaba automáticamente a un entorno de pruebas.

El chef no necesitaba estar en la cocina para que se hiciera el pedido. El robot cocinaba por él.

Paso 5: Abrir el restaurante en la nube (AWS)

Con ECS, deployaron contenedores de ARKA. Usaron S3 para archivos, RDS para la base de datos, CloudWatch para logs y variables de entorno para seguridad. Todo estaba bajo control y escalaba solo.

Paso 6: Trabajar como un solo equipo (DevOps)

No había diferencia entre Dev y Ops. Todos sabían cómo funcionaban los entornos, cómo se monitoreaban los errores, cómo se hacía rollback. Nadie decía "eso no es mío". La cocina y los meseros eran uno solo.

Resultado final

ARKA fue desplegada exitosamente con un backend mantenible, probado, documentado, automatizado y desplegado en la nube. Cada decisión técnica tenía un "por qué" y un "cuándo sí/cuándo no" claro.

Así, ARKA se convirtió en mucho más que una app: fue una clase viva de backend bien hecho.

Epílogo

Si estás leyendo esto, ya tienes en tus manos las herramientas para hacer lo mismo. No necesitas saberlo todo desde el primer día, pero sí saber **cuándo aplicar cada cosa, por qué, y cómo integrarlas**. Esa es la diferencia entre saber programar... y ser un verdadero backend developer.

¡Buen viaje, cocinero del backend!